A SOURCE-GROUNDED WALKTHROUGH

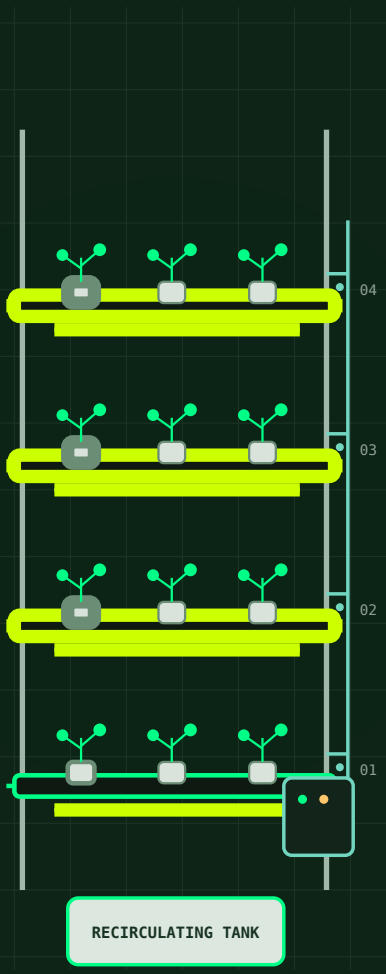
PROJECT VERDE

The idea, the system, the code, and the honest boundary between them.

A detailed walkthrough for presenting an autonomous vertical-growing concept and the interactive Next.js experience that documents it.

WHY IT EXISTS · WHAT IT USES · WHAT IS REAL

CONCEPTUAL FOUR-TIER TOWER



NOT TO SCALE · ILLUSTRATIVE

THE CENTRAL IDEA

Sense locally. Decide intelligently. Reuse the water.

Report assembled from the checked repository snapshot · 18 October 2026

NAVIGATION / HOW TO USE THIS GUIDE

Contents

Read the first five chapters for the project story and system boundary. Chapters 06–14 explain the hardware concept, website, 3D architecture, APIs, and evidence. The final chapters prepare a clear presentation, answer examiner questions, and give a practical next-step checklist.

Project Verde in one page 3
Why I made Project Verde 4
What the repository really contains 5
From plant to screen 6
What the tower is meant to sense 7
How sensing becomes a physical action 8
Local decisions, cloud support 9
What the website actually uses 10
How the page tells the story 11
From narrative to inspectable detail 12
Three scenes, one persistent lifecycle 13
What each endpoint returns 14
What to say carefully 15
Where to find each part 16
Why the experience feels the way it does 17
How to reproduce the website 18
A clear four-minute talk track 19
Questions about the growing system 20
Questions about the code 21
What to improve next 22
Terms and where the facts came from 23

HOW TO INTERPRET THE EVIDENCE
This walkthrough separates what the current repository implements from what the project narrative describes. It is a source review, not an independent test of a physical tower. The first-person motivation and speaking script are suggested framing inferred from the project's own manifesto; adjust them to match your actual experience.

01 / EXECUTIVE SNAPSHOT

Project Verde in one page

A systems-engineering story about growing vertically, sensing the root zone, and making irrigation decisions with less manual attention.

THE ONE-SENTENCE VERSION

Project Verde is described as a four-tier, twenty-site recirculating hydroponic tower: sensors observe plants and the growing environment, an ESP8266-based controller is intended to make local decisions, actuators manage water and light, and a Next.js experience makes the design understandable to an audience.

4	20	ESP8266	3
STACKED TIERS	GROW SITES	EDGE CONTROLLER	INTERACTIVE 3D SCENES

Two linked deliverables

LAYER	WHAT IT CONTRIBUTES	STATUS IN THIS REPOSITORY
Growing-system concept	Hydroponics, sensors, embedded control, edge vision, power electronics, recirculation, cloud messaging.	Described in page copy and diagrams; firmware and hardware design files are not included here.
Interactive website	A scroll-driven technical tour with 3D scenes, animations, API demo routes, and responsive presentation.	Implemented in Next.js/React/TypeScript; the home page is statically generated.
Data layer	Telemetry, weather, system health, statistics, and contact-style endpoints.	Route handlers exist, but most return sample/static values; the UI's live-looking displays are not connected to them.

The central design idea is integration—not a single sensor or model. The value is in connecting observation → decision → safe action → feedback, then explaining that chain clearly.

Repository evidence: current page composition and metadata (S2, S6); project narrative and claims (S1).

02 / PURPOSE

Why I made Project Verde

A presentation-ready rationale drawn from the project’s stated themes: water efficiency, compact growing, and useful automation.

The problem the project chooses to address

The site’s manifesto frames three connected pressures: limited arable space, pressure on water resources, and loss between harvest and plate. Rather than treating them as separate talking points, Verde proposes one small, controlled growing system that uses vertical space and returns nutrient solution to a reservoir. That is the project’s problem statement—not a claim that one tower can solve food security on its own.

A first-person explanation you can adapt

> **SUGGESTED PERSONAL FRAMING**
> “I made Project Verde because I wanted to explore how a compact growing system could use water and space more carefully, and how sensors could reduce the guesswork of looking after plants. I wanted it to be more than a model of a tower: it should connect sensing, embedded control, edge intelligence, recirculation, and a clear way to inspect the idea. The website is part of that work—it lets someone follow the system from a plant’s root zone to the controller and back to the reservoir.”

Why these design choices fit the problem

CHOICE	WHY IT HELPS THE PROJECT QUESTION	TRADE-OFF TO ACKNOWLEDGE
Vertical hydroponics	Stacks growing sites and recirculates a shared nutrient solution.	Requires pump, lighting, nutrient balance, cleaning, and leak/failure planning.
Per-site sensing	Responds to a plant’s root-zone condition rather than a single timer for an entire shelf.	Sensors need calibration; more channels increase wiring and fault-detection work.
Local decisions	The intended controller can keep core actions running when Wi-Fi is unavailable.	Small microcontrollers have tight RAM, flash, and compute budgets.
Interactive web guide	Makes invisible engineering legible to students, judges, and visitors.	A polished simulation can look more “live” than the underlying implementation; label sample data.

The project is also an educational integration exercise: biology, electronics, embedded software, networking, AI, industrial design, and communication have to agree with one another.

The exact personal motivation cannot be proven from source code. The wording above is a strong draft based on the repository’s own manifesto; replace it if your real reason was different.

03 / SCOPE AND EVIDENCE

What the repository really contains

This distinction matters: a concept page can describe a complete IoT system even when the repository only ships the website and demo endpoints.

AREA	WHAT IS PRESENT HERE	WHAT IS NOT PRESENT HERE
Web app	Next.js App Router page, React components, TypeScript, Tailwind/custom CSS, animation and navigation components.	No separate mobile app, authentication flow, admin console, or deployed-device pairing UI.
3D	Three standalone HTML scenes under public/scenes; vendored Three.js runtimes and controls/add-ons.	No GLB/GLTF model asset is in the repo; the active scenes construct their geometry in code.
APIs	Five Next.js route directories: contact, health, stats, telemetry, weather.	No Firebase SDK/client, Twilio SDK/webhook, OpenWeatherMap fetch client, or persistent storage implementation.
Embedded/AI	Architecture and performance claims in UI copy, README, FAQ, and model prompts.	No .ino/.cpp firmware, KiCad schematic/PCB, TFLite file, training dataset, pin map, or sensor log is included.
Contact	POST handler checks required fields and a basic email shape, then returns a generated ID.	No email delivery, ticket creation, database write, abuse prevention, or user-facing contact form is wired into the page.

A useful evidence vocabulary

LABEL	USE IT WHEN…	EXAMPLE FROM THIS PROJECT
Implemented	The behavior is in the source and can be demonstrated from the site.	Responsive page, keyboard command palette, procedural Three.js viewer, route response.
Described	The source explains an intended physical feature, but supporting engineering files are elsewhere or absent.	ESP8266 firmware, custom KiCad board, TFLite classifier, WhatsApp integration.
Simulated/sample	Values or activity are generated for presentation rather than read from a sensor/service.	Telemetry random walks, log pool, sample forecast, nominal health checks.
Verified result	A reproducible measurement/test exists with method, baseline, and evidence.	No experiment logs or test report are included for the advertised savings/accuracy figures.

DO NOT OVERSTATE WHAT THIS CODE PROVES

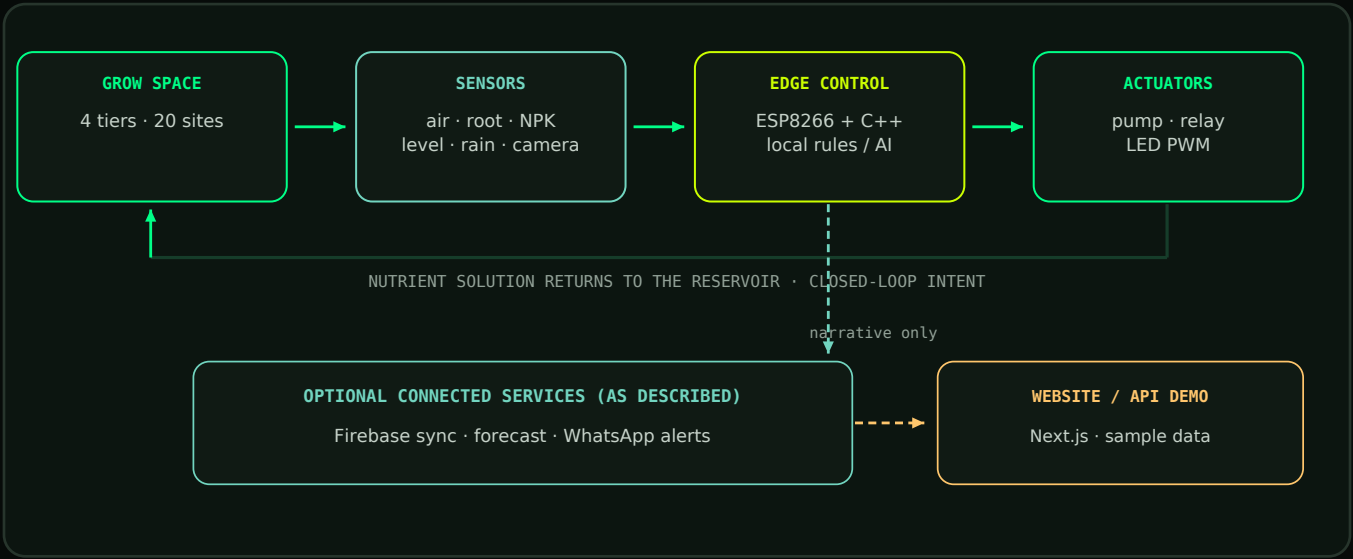
The repository proves that the interactive website builds and that demo routes exist. It does not, by itself, prove that a physical tower is operating, that a cloud account is connected, that a classifier achieved a particular accuracy, or that a water-saving percentage was measured. Those may exist outside this repository; show the separate evidence if you have it.

Source review checked active files and searched for firmware, PCB, model, mesh, and CSV artifacts; none of those file types is present in the repository snapshot. See the end-of-guide source map.

04 / SYSTEM ARCHITECTURE

From plant to screen

Separate the physical control path from the web presentation path. They are related in the story, but they are not currently wired together in code.



How the intended physical loop works

STEP	SYSTEM ROLE	EXPLANATION
1 · Observe	Sensors sample the grow environment.	Root moisture, air temperature/humidity, nutrient readings, reservoir level, rainfall, and canopy images are the kinds of inputs described by the site.
2 · Interpret	The controller establishes state.	The ESP8266 firmware is described as sampling/handling sensor values and keeping a local state for each growing site.
3 · Decide	Rules and optional vision inform action.	The narrative describes threshold logic, forecast-aware deferral, and on-device leaf-health classification. Actual firmware is not in this repo.
4 · Act	Outputs control hardware.	A relay is described for the pump; MOSFET PWM channels are described for the grow-light array.
5 · Close the loop	Water returns to the tank.	The hydroponic fluid path is designed to recirculate solution, reducing waste relative to an open drain-to-soil baseline.
6 · Report	Cloud and web communicate state.	Firebase/WhatsApp/weather are named in the concept. The current website shows a demo narrative; it does not fetch device data.

IMPORTANT ARCHITECTURE BOUNDARY
The diagram’s green path is the intended plant-control loop. The dashed cyan/amber path is supporting connectivity and the website’s presentation layer. The current browser UI does not call the API routes or subscribe to Firebase; the telemetry and terminal are locally simulated in their React components.

This separation is a strength if explained clearly: a small controller can preserve local control while the cloud improves reporting and remote access. It also means the web demo must not be presented as a live view of the real tower unless the integration is added.

05 / HARDWARE · SENSING

What the tower is meant to sense

The sensor list comes from active site content and the README. The list below describes purpose and signal type, not a measured wiring diagram.

SENSOR / INPUT	WHAT IT TELLS THE CONTROLLER	PROJECT-SITE DETAIL / CAVEAT
Capacitive root-zone probe(s)	How wet the growing medium/root zone is at an individual site; supports plant-specific irrigation.	Specs page labels the probe capacitive/analog. The sensor-pod illustration shows a two-prong probe. Confirm the real probe type before a hardware talk.
DHT22	Air temperature and relative humidity around the canopy.	Listed in the subsystem explorer; use calibrated readings and sensor placement away from direct water/light heat.
HC-SR04 ultrasonic	Distance to the reservoir surface; converted to level or remaining volume.	Site says about ±3 mm. That is a stated design value, not a test result here; geometry and surface motion affect accuracy.
NPK RS485 / Modbus probe	Approximate nutrient indicators for nitrogen, phosphorus, and potassium.	The project calls out an RS485 bus. Explain calibration and solution compatibility; raw ppm values need a sensor-specific method.
Rain sensor	Rain/wetness signal to suppress an unnecessary watering event or serve as a fallback.	A rain sensor is listed; the FAQ describes a forecast fallback relationship. It is not connected in this web code.
OV2640 camera	Canopy image used by the described leaf-health/disease model.	The site calls it a 2 MP camera. The repo has no camera driver, sample images, model, or inference pipeline.

Why sensing must be designed as a system

- A threshold is only useful after calibration: record the sensor output in known dry, normal, and wet conditions, then set limits for the actual plant/crop.
- Treat sensor failure as a state, not as a reading of zero. A disconnect should not trigger uncontrolled irrigation.
- Measure at the root zone where possible. One shelf-level reading can hide a dry pot next to an over-watered one.
- Log units, timestamps, sensor identity, calibration date, and quality flags. That is what turns a dashboard number into evidence.

NARRATIVE SPEC TO RECONCILE

The scroll deck says twelve channels at 0.5 Hz; other modules describe different sampling cadences (for example, a reservoir reading every 30 seconds). Treat these as intended/displayed specs until one firmware configuration and timing table is available.

06 / HARDWARE · COMPUTE AND ACTUATION

How sensing becomes a physical action

The project describes a small Wi-Fi microcontroller, a custom power/control board, and two visible outputs: water circulation and grow lighting.

SUBSYSTEM	WHAT THE PROJECT DESCRIBES	WHY IT MATTERS
ESP8266 NodeMCU	160 MHz Tensilica L106, 4 MB flash, Wi-Fi; single controller for sensing, relay, PWM, and uplink.	Low-cost, compact, familiar embedded platform with built-in Wi-Fi. Exact board revision and RAM budget should be documented.
Firmware	1,200+ lines of C++; non-blocking scheduler, debounced sensor sampling, safety interlocks, watchdog.	A non-blocking loop keeps sensing, UI-free control, and actuator timing responsive. No source is included for review.
Custom PCB / KiCad	Two-layer board with LM2596 buck conversion, flyback-protected relay path, screw terminals.	Moves beyond breadboard wiring, makes power distribution and field wiring more repeatable. No schematic/layout files are included.
Pump / relay	2.4 L/min recirculating pump switched through an SPDT relay.	Moves nutrient solution. Requires current-rating margin, isolation, dry-run detection, leak containment, and a timed failsafe.
Grow lights	Twelve LEDs per tier, 380–780 nm, controlled through MOSFET PWM channels.	Supports repeatable lighting across stacked shelves. Validate spectrum, heat, current, and actual photoperiod for the crop.

The power path is a safety story

A pump and a lighting array are electrical loads, not just UI toggles. A real build should show its supply voltage/current, fuse or current-limiting strategy, grounding/isolation, relay contact rating, flyback protection, connector ratings, enclosure, and separation between water and mains. The site mentions an LM2596 buck and flyback protection; the underlying schematic and test evidence are not in the repository.

Safety behavior worth demonstrating

FAILURE	SAFE RESPONSE TO DESIGN FOR
Sensor disconnected / implausible	Mark data invalid; stop or constrain watering; raise a local fault.
Wi-Fi or cloud lost	Continue bounded local rules; buffer logs if safe; show connectivity loss rather than fake success.
Pump relay stuck / dry reservoir	Use a maximum run time, low-level interlock, watchdog, and physical emergency cutoff.
Reset / brownout	Default outputs to safe states; restart predictably and record the reset reason.

EMBEDDED AI MEMORY CHECK

The page narrative gives a quantized TFLite model of 612 KB and a 214 KB inference arena, and elsewhere gives a 38 ms inference figure. Verify these numbers against the exact ESP8266 board, available RAM, model binary, interpreter build, and measured benchmark. The repository contains none of those artifacts, so the figures should be described as stated specs—not independently confirmed performance.

07 / SOFTWARE INTELLIGENCE

Local decisions, cloud support

A clean explanation is to make the local control loop primary and describe network services as optional reporting and context—not as the controller's only brain.



CAPABILITY	PROJECT'S INTENDED ROLE	REPOSITORY REALITY
Rule-based irrigation	Use plant/root state and reservoir/weather context to decide when to water.	Described across the tour, FAQ, and timeline; no embedded rules source or device test is included.
TensorFlow Lite	Run a quantized canopy classifier near the camera for early leaf-health/disease signals.	Named in the site and README; model file, training set, preprocessing code, and evaluation report are absent.
Firebase RTDB	Mirror telemetry and potentially deliver commands/status to a dashboard.	No Firebase package or configuration appears in package.json or active app code; website data is not read from Firebase.
Twilio WhatsApp	Send alerts and answer questions in English/Hindi through a familiar channel.	Only mentioned in narrative and mock log/boot text; no Twilio SDK, webhook, or credentials are present.
OpenWeatherMap	Use forecast context to defer an irrigation cycle before rain or extreme heat.	The weather route returns fixed example data; it does not make a network request to OpenWeatherMap.

Why use both edge and cloud?

Edge execution can keep a local rule running without internet, reduce latency, and avoid sending raw plant imagery away. Cloud services can support remote monitoring, long-term history, forecast retrieval, and human notifications. Those are complementary roles. A forecast lookup still needs connectivity, so "no cloud dependency" should mean that basic safe irrigation can continue without cloud—not that the entire system has no external dependency.

> **PRIVACY AND RESILIENCE ARE USEFUL TALKING POINTS**
> If the camera model truly runs on-device, explain what images are stored, whether any leave the tower, how long they are retained, and what happens if the model is unavailable. If the connection drops, state which functions continue locally and which (forecast sync, remote alerts, dashboard updates) pause.

A concise spoken architecture: "The controller makes local, bounded decisions; a network link adds history and helpful context. If that link disappears, safety-critical local behavior should still be predictable."

08 / WEBSITE IMPLEMENTATION

What the website actually uses

The active package manifest and imports are the source of truth. A few technology names in README.md describe an older or aspirational version of the site.

LAYER	ACTUAL TOOLING IN THIS CHECKOUT	WHAT IT DOES HERE
Framework / runtime	Next.js 14.2.5 App Router · React 18.2 · TypeScript 5.3	One route composition, static rendering, metadata, and serverless route handlers.
Styling	Tailwind CSS 3.4 · PostCSS · Autoprefixer · app/globals.css	Utility layout plus custom tokens, effects, typography, focus styles, and reduced-motion CSS.
Motion / scroll	Framer Motion 11 · Lenis 1.3	Scroll-linked storytelling, entrance transitions, spring values, counters, and optional smooth scrolling.
3D	Vendored Three.js r162 modules + legacy r128 global build; OrbitControls and selected add-ons.	Runs in plain static HTML scenes within iframes; not part of the npm dependency graph.
Icons / fonts	Lucide React · Fontsource: DM Sans Variable, Manrope Variable, Oswald Variable, IBM Plex Mono.	Lucide supplies UI icons; local font packages load the main site’s typography.
Build / deploy	Node 20 target · npm ci · Vercel · region bom1 (Mumbai).	Standard Next build; no custom server or adapter is configured.
Dev tools	TypeScript, ESLint / eslint-config-next, Tailwind, PostCSS, rimraf, React/Node type packages.	Type-checking, linting, styles, and clean local dev startup.

What is not in the active stack

- React Three Fiber and Drei are not installed or imported. The 3D scenes use Three.js directly in their own HTML documents.
- There is no Firebase, Twilio, TensorFlow/TFLite, OpenWeatherMap, charting, or sensor SDK dependency in package.json.
- There is no test runner or test script. The repository’s `verify` script is typecheck + Next lint + production build.
- Main-site fonts are Fontsource packages, not Space Grotesk / JetBrains Mono. Those names in README are stale.

DOCUMENTATION DRIFT FOUND

README.md lists components, hooks, sections, an Easter egg, React Three Fiber/Drei, and fonts that do not match the current tree. The active code has one App Router page composed from app/components/*.tsx and uses direct Three.js iframes. Update the README before treating it as an implementation inventory.

Package manifest and active imports reviewed: package.json, app/layout.tsx, app/page.tsx, app/components/ScenePreloader.tsx, public/scenes/*.html (S2, S3, S4, S6).

09 / WEBSITE WALKTHROUGH · OPENING

How the page tells the story

The home route is a long, deliberately paced scrollytelling experience—not a multi-page product dashboard.

ORDER / COMPONENT	WHAT THE VISITOR SEES	WHY THIS SECTION EXISTS
BootCurtain · ScenePreloader	A terminal-style startup screen while three 3D iframes initialize; progress and log lines animate.	Creates an exhibit-like opening and avoids showing uninitialized canvases. The log text is a visual effect, not a live hardware boot log.
Hero	“WE GROW THE FUTURE”; status HUD; CTA; particle/neural field; sample readouts.	Introduce the project and prompt a guided tour. Flow is randomized after mount; displayed sensor cards are fixed sample values.
Velocity marquees	Moving word bands (AUTONOMOUS, SEALED LOOP, ON-DEVICE AI, etc.) that react to scroll direction/velocity.	Provide a visual transition while reinforcing the key vocabulary.
Manifesto	“SOIL IS RUNNING OUT” with context about land, water, and food waste.	Make the problem legible before presenting the engineering response.
StatsStrip	Large counters for water reclaimed, twenty sites, and zero cloud dependency.	Summarize the concept at a glance. The values are hard-coded display claims, not fetched measurements.
Pillars	Three cards: self-watering, plant watching, communication; hover tilt and spotlight.	Explain the system in plain language before showing its internal steps.

A useful visitor transition

> **THE STORY LOGIC**
> Problem → promise → proof shape. First establish a real-world motivation, then name the intended capabilities, and only then expose the stages and parts. The visitor should understand why a sensor or PCB exists before being asked to memorize its model number.

The composition lives in app/page.tsx. It imports reusable section components and fixes their order; the page file itself stays concise. The navigation in app/components/site/Nav.tsx links to section anchors, while SideRail.tsx highlights a section on larger screens.

The page title and metadata are defined in app/layout.tsx; SEO endpoints are generated by app/robots.ts and app/sitemap.ts. The website is designed as both the exhibit’s front door and its technical compendium.

10 / WEBSITE WALKTHROUGH · ENGINEERING

From narrative to inspectable detail

The middle and closing sections move from “what it does” to “how it works,” then leave the visitor with facts, caveats, and people.

SECTION	WHAT IT DOES	WHAT TO SAY WHILE PRESENTING
HorizontalDeck · #loop	Six pinned, scroll-driven cards: SENSE, SEE, DECIDE, ACT, REPORT, RECLAIM.	This is the project’s full closed-loop mental model; scroll slowly so each stage has time to read.
TowerStory · #tour	Four sticky chapters; the camera of the tower scene moves to canopy, vision, root zone, and base.	Use the model as a visual pointer. The text explains the hardware/data story; it is not a live sensor camera.
LiveTerminal · #console	A continuously updating console from a fixed pool of example events.	It illustrates event categories—INFO, WARN, EXEC, AI. The timestamps are real browser time, but the event messages are scripted.
Telemetry · #telemetry	Water temp, pH, humidity, EC sparklines, and an 82% reservoir gauge.	Read it as a UI prototype: the random walk is client-side; the values do not come from the API or Firebase.
Comparison · #versus	Animated soil-vs-Verde bars for water, area, harvest time, intervention, runoff.	Explain the proposed comparison baseline. The bars/numbers are constants in the component; no study dataset is included.
SensingSection · #sensing	One seedling/pod scene with sample moisture, root temperature, and last-fed cards.	Show how the concept aims for per-plant decisions, while labeling these readouts as examples.
BoardSection · #hardware	A controller-board scene; the parent page can request spin/explode actions.	Point from MCU to protection/power and load control; the scene is illustrative, not a PCB design file.
SpecExplorer · #specs	Tabbed cards for Sensing, Compute, Power, Cloud.	Use it as the technical vocabulary page. State what is an intended spec versus a tested value.
Timeline · #build / FAQ / Team	Build stages, five expandable questions, two named contributor cards, and a closing CTA.	Explain the development sequence and split of responsibilities; invite questions at the end.

Navigation shortcuts

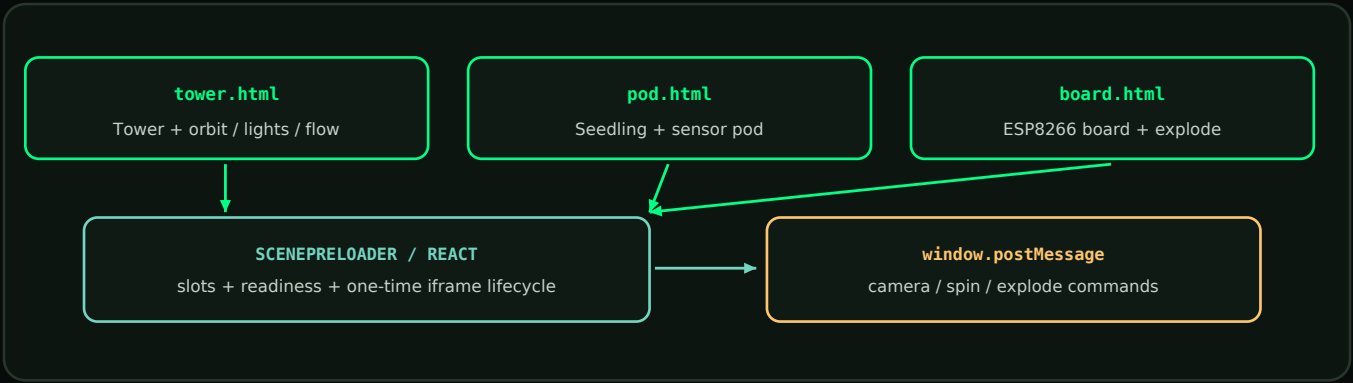
The desktop navigation, XL side rail, footer columns, and command palette all jump to named anchors. The command palette opens with Ctrl+K / Cmd+K and supports Arrow Up/Down, Enter, and Escape. On small screens the top navigation switches to a menu button; the command-palette button is hidden on mobile.

Interaction pattern: most section entrances use Framer Motion `whileInView` once; selected headlines use SplitText or TextReveal; only scroll-linked sections read scroll progress continuously. That keeps the page expressive without turning every paragraph into an animation.

11 / 3D AND INTERACTION ARCHITECTURE

Three scenes, one persistent lifecycle

The Three.js scenes are independent documents. React positions and commands their iframes; the geometry and rendering loops live inside the HTML files.



SCENE	ACTIVE FILE	INTERACTIONS / DETAILS
Tower	public/scenes/tower.html	Procedural four-tier tower; OrbitControls; rotate, lights, water-flow, reset. React TowerStory sends camera `view` commands as the visitor scrolls.
Plant pod	public/scenes/pod.html	Procedural seedling, pod, probe/board details, callouts, selectable views, orbit and zoom; reports first render to parent.
Controller board	public/scenes/board.html	Procedural NodeMCU-style board; hover inspector, orbit, LED/Wi-Fi toggles, exploded view, top/reset; parent can send spin/explode.

Why the persistent iframe approach?

ScenePreloader mounts all three iframes at page load, listens for each `scene-ready` message, and exposes SceneSlot placeholders. A requestAnimationFrame loop moves each fixed-position iframe wrapper over the closest visible slot. This lets a WebGL scene keep its context and avoids rebuilding/re-parsing the tower every time a visitor scrolls away and back. Scene messages use the `verde: 'cmd'` shape and browser `postMessage`.

The boot curtain waits for the scenes (or an 11-second escape timeout), observes a 2.6-second minimum, then fades after readiness. The percentage is derived from scene count; memory and scripted boot messages are decorative. The tower’s camera chapters are driven by Framer Motion scroll progress; the board’s spin/explode is also commanded from React.

> **VERSION AND ASSET DETAILS**
> Tower and pod import the locally vendored Three.js module build at r162 plus selected add-ons; the board uses a legacy global Three.js r128 build. The three active models are made from geometry/material code, not loaded GLB assets. The root model1.html/model2.html/model3.html files are standalone prototypes and are not imported by app/page.tsx. Tower/pod HTML also contain Google Fonts CSS imports; local fallback fonts keep the scene legible if those fonts cannot load.

The scene files have their own renderers and animation loops. Three WebGL contexts can be demanding on low-end hardware; the preloader’s convenience trades against startup GPU work. The CSS and component code include some reduced-motion accommodations, but performance should be tested on the actual exhibit device.

12 / API AND DATA BEHAVIOR

What each endpoint returns

These are real Next.js handlers, but “endpoint exists” is not the same as “service is integrated.” The routes are not called by the current page components.

ROUTE	METHOD / BUILD	CURRENT BEHAVIOR	DATA STATUS
/api/health	GET · static in inspected build	Returns HEALTHY plus nominal database/API/ML/mesh/storage/weather fields; random-looking latency values are evaluated at build time in the current static output.	Demo snapshot; no probes to real services.
/api/stats	GET · static	Returns fixed sample plant, irrigation, water, uptime, traffic, user, city, and cost values.	Hard-coded example payload.
/api/weather	GET · static	Returns a Delhi location, current weather, five forecast rows, recommendation, and historical series.	Fixed sample; no upstream weather fetch.
/api/telemetry	GET · dynamic	Generates randomized moisture/temp/humidity/light/tank/NPK/actuator/wifi values and short trends.	Synthetic per request.
/api/telemetry	POST · dynamic	Parses JSON, echoes it back, and labels the result `QUEUED`.	No queue, database, auth, or device validation.
/api/contact	GET/POST · dynamic	GET returns endpoint status. POST checks name/email/message and returns a generated transmission ID + acknowledgement.	No email or persistence; not wired to a page form.

Deployment rendering confirmed by `npm run verify` during this review

STATIC / PRERENDERED	DYNAMIC / ON-DEMAND
/ · /robots.txt · /sitemap.xml · /api/health · /api/stats · /api/weather	/api/contact · /api/telemetry
Good for fast CDN delivery; static endpoint payloads are snapshots, not live probes.	Serverless/dynamic route behavior; still sample logic unless connected to actual services.

PUBLIC API HARDENING BEFORE REAL DEPLOYMENT

next.config.mjs sets wildcard CORS headers for /api/*. That is acceptable for a public demo but should be narrowed for real telemetry. Add input schemas, authentication/device identity, rate limiting, request-size limits, server-side secret storage, audit logging, and explicit preflight behavior before accepting real devices or personal contact data.

The route code is in app/api/*/route.ts; endpoint headers are in next.config.mjs. The web UI does not fetch `/api/*` today, so opening the page will not make these routes live.

13 / CLAIMS AND LIMITATIONS

What to say carefully

A strong technical presentation is specific about what is measured, what is a design target, and what is currently a visual simulation.

DISPLAYED CLAIM / FIGURE	WHERE IT APPEARS	WHAT NEEDS RECONCILING
95% less water / 95% reclaimed	Hero/story copy, StatsStrip, comparison bars, FAQ, tower story.	The comparison page claims same 20 plants, same Delhi conditions, three cycles, with 18.0 L vs 0.9 L. The API stats endpoint instead says waterSaved 42%. No measurement log is included. Define “saved” vs “reclaimed,” baseline, cycle, and test evidence.
TFLite · 612 KB · 214 KB arena · 38 ms	HorizontalDeck / BootCurtain narrative.	The health route says 340 ms and 91.3% accuracy; README also gives 91.3%. Verify the actual model, runtime, board RAM, image pipeline, dataset, benchmark, and validation split. The numbers conflict.
Zero cloud dependency / 0 ms cloud	StatsStrip and loop deck.	The same project describes cloud sync, Twilio alerts, and an OpenWeatherMap forecast. Better wording: local basic control is designed to continue without cloud; cloud features add reporting/context.
Live telemetry / real event stream	Telemetry section and LiveTerminal.	Telemetry uses React-side seeded random walks; the console cycles through a fixed event pool. The API also generates synthetic values, and neither is read by the UI.
Sensors sampling every two seconds	HorizontalDeck / spec copy.	Other text says reservoir sampling every 30 seconds. Publish one timing table, identify channels, and make sampling/cadence claims match firmware.
Four tiers / 20 sites / zero human intervention	Hero, manifesto, footer, description.	These are project configuration/marketing claims. Show an actual assembled tower, capacity definition, and which tasks still require refill, cleaning, nutrient adjustment, or harvest.

Presentation-safe phrasing

> **SEPARATE TARGET FROM PROOF**
> Say: “The design target is around 95% less water than our stated soil-bed comparison. The website visualizes that target; this repository does not contain the measurement log. For a validated result, I would show metered litres across the same crop, duration, and growing conditions.”

For every numerical claim, prepare the measuring instrument, units, sampling interval, baseline, number of runs, uncertainty, and raw data. If a number is only an illustrative UI value, label it clearly as a demo value rather than presenting it as a sensor reading.

The site’s product narrative is ambitious and cinematic by design. A jury will trust it more when the demonstration states exactly where the simulation ends and the tested system begins.

14 / REPOSITORY TOUR

Where to find each part

A source-first tour makes the code easier to defend. Start at the route composition, then trace the components and static scenes it pulls in.

```
app/
page.tsx Single-page order / composition
layout.tsx Fonts, metadata, global CSS, HTML shell
globals.css Theme tokens, Tailwind layers, motion effects
robots.ts / sitemap.ts Search crawler metadata
api/ Five App Router route handlers
components/ Page sections + reusable interaction modules
site/Nav.tsx Anchor navigation + mobile menu + command palette
site/Footer.tsx Footer navigation and project identity
ScenePreloader.tsx Three iframe lifecycle, slots, postMessage bridge
BootCurtain.tsx Visual startup animation (scripted diagnostics)
Hero.tsx / NeuralField.tsx Landing and canvas background
HorizontalDeck.tsx Six-step pinned scroll narrative
TowerStory.tsx Four chapters / camera movement
Telemetry.tsx Client-side animated demo charts
LiveTerminal.tsx Scripted event pool / scrolling console
SpecExplorer.tsx Sensing, compute, power, cloud tabs
... Manifesto, comparison, FAQ, team, etc.
public/
scenes/ Standalone Three.js HTML documents
vendor/ Local Three.js builds + controls/add-ons
favicon.svg / manifest.json
google*.html Search verification files
docs/
3d-model-prompts.md Prompts for possible generated models, not model assets
model1.html / model2.html / model3.html Root-level 3D prototypes; not page imports
package.json / package-lock.json Dependencies, scripts, exact versions
next.config.mjs / vercel.json Headers, build, caching, deployment region
README.md / DEPLOYMENT.md Narrative overview / deployment notes
```

Key reading order for a code review

START HERE	FOLLOW THE IMPORT / DATA FLOW
1 · app/page.tsx	See the actual section order and each component imported by the active home route.
2 · app/components/ScenePreloader.tsx	Understand first-frame readiness, the boot curtain, scene slots, and iframe command bridge.
3 · one section component	Read the copy and find whether its values come from constants, local state, or an API. Most page values are constants/local animations.
4 · app/api/*/route.ts	Inspect exact route behavior; distinguish fixed/random demo payloads from real service adapters.
5 · package.json + configs	Confirm the actual dependencies/scripts and how the app builds/deploys.

LEGACY FILES TO RECOGNIZE

The root `model1.html`, `model2.html`, and `model3.html` are older standalone 3D documents that refer to CDN builds. The current Next page instead embeds `public/scenes/tower.html`, `pod.html`, and `board.html`, which use vendored Three.js files. The docs prompt file describes desired generated models; it is not evidence that GLB files were added.

15 / DESIGN, MOTION AND ACCESSIBILITY

Why the experience feels the way it does

A restrained dark system UI, oversized editorial typography, and motion make the project feel like an instrument panel without hiding the story.

DESIGN CHOICE	IMPLEMENTATION	INTENT
Palette	Near-black background; bright emerald `#00FF87`; lime `#CCFF00`; hairline white borders.	Make plant/energy accents legible while leaving the 3D objects as focal points.
Type	DM Sans body; Manrope display token; Oswald variable for `.font-display-xl`; IBM Plex Mono for readouts.	Mix readable editorial copy with industrial/technical labels.
Layout	Tailwind responsive grids, sticky sections, card surfaces, HUD frames, moving marquees.	Give the long page clear rhythm and a consistent “system console” language.
Motion	Framer Motion scroll values, springs, while-in-view reveals, SVG gauges; custom CSS scanline/glitch; Canvas particle lattice.	Show time, flow, progression, and focus. Use motion to explain relationships, not as decoration alone.

Accessibility and inclusive interaction

- Global `:focus-visible` provides a visible emerald outline; key controls have labels/expanded states; FAQ and mobile menu expose `aria-expanded`.
- Decorative canvas and visual indicators are hidden from assistive technology where appropriate; headings use accessible labels when text is split into animated spans.
- Lenis and the 2D particle field check `prefers-reduced-motion`; global CSS shortens transitions, shows reveal content, and disables some effects.
- The 3D scenes expose orbit controls and some buttons/labels, but interactive 3D is not a substitute for a text description of every component.
- Custom cursor only activates for fine-pointer/hover devices; keyboard users should still have native focus and click behavior.

Performance trade-offs to understand

The `/` route is statically generated, and static scene/vendor files can be cached. The Three.js code is locally vendored instead of being fetched from a script CDN. However, all three iframe scenes are mounted at load and each scene has an animation loop. That improves scroll continuity but can increase WebGL memory/CPU/GPU work. Verify on the device used for the exhibition; consider lazy scene startup, pausing offscreen loops, a still-image fallback, and mobile quality caps.

Deployment headers include immutable one-year caching for `/vendor/*`, daily revalidation for `/scenes/*`, SAMEORIGIN framing for scenes, plus `nosniff` and a strict referrer policy. The two ES-module scenes also contain Google Fonts CSS imports with local fallbacks; they may make an external font request when online.

16 / RUN, VERIFY AND DEPLOY

How to reproduce the website

The project is a standard Next.js 14 App Router application. Use the pinned Node version from `.nvmrc` for the closest match to deployment.

```
# Node 20 is specified by .nvmrc
npm ci
npm run dev # development server
npm run typecheck # TypeScript only
npm run lint # Next ESLint
npm run build # production build
npm run verify # typecheck + lint + build
npm start # serve a built app locally
```

CONFIGURATION	CURRENT SETTING / EFFECT
Node	`.nvmrc` says 20; package.json allows >=18.18. Vercel setup notes use Node 20.x.
Vercel	vercel.json pins Next.js, `npm ci`, `next build`, and region `bom1` (Mumbai).
Environment	No env variable is required for the demo. `NEXT_PUBLIC_SITE_URL` is optional for canonical/metadata URLs; Vercel URL variables are fallback sources.
Static assets	Scenes are under `public/scenes`; Three files are under `public/vendor`; the static HTML is served same-origin.
No custom server	No adapter/custom server is configured; standard Next build and route behavior.

Verification performed for this walkthrough

On 18 October 2026, `npm ci` completed and `npm run verify` passed in the supplied checkout: TypeScript check, Next lint, and production build all completed. The command ran here on Node 22.22.3; `.nvmrc` specifies Node 20, so repeat the verification on Node 20 before assuming identical deployment behavior. The build output marked `/` and health/stats/weather as static, and contact/telemetry as dynamic. No unit, API-contract, browser, or hardware-in-the-loop tests are defined in package.json.

SECURITY MAINTENANCE FLAG FROM THE SAME INSTALL
The lockfile audit reported 21 dependency advisories (2 moderate, 18 high, 1 critical); `npm audit --omit=dev` reported 4 production dependency findings (3 high, 1 critical). The direct Next.js dependency is 14.2.5; npm audit identified 14.2.35 as a non-major fix target. This report does not change dependencies—schedule an audited patch upgrade, review the lockfile, then rerun verify and audit before a public production release.

`npm ci` also printed deprecation/security notices. Build success is a correctness check for compilation, not a security clearance or proof that cloud/hardware services work.

17 / PRESENTATION-READY WALKTHROUGH

A clear four-minute talk track

Use this as a rehearsal draft, not a script you must memorize. Keep claims aligned with what you can physically demonstrate and measure.

REHEARSAL DRAFT · ABOUT FOUR MINUTES

Good morning. I'm Anuj Phulera, and I worked with Aarav Choudhary on Project Verde. I made this project to explore a practical question: how can a compact growing system use space and water more carefully, while asking a person to do less routine monitoring?

Verde is described as a four-tier hydroponic tower with twenty growing sites. Instead of watering everything on a timer, the idea is to read the root zone, air, nutrient condition, reservoir, weather context, and plant canopy. An ESP8266-based controller is intended to combine those signals, make bounded local decisions, and operate a pump and grow lights. Nutrient solution returns to the reservoir, which is the closed-loop part.

The project also describes an on-device TensorFlow Lite vision model, so canopy checks could happen close to the camera, and cloud services for telemetry and alerts. Local control and cloud reporting have different jobs: core safe behavior should continue if the network drops, while remote history and notifications can pause.

The website is part of the project, not just a brochure. It is built with Next.js, React, TypeScript, Tailwind, Framer Motion, and locally vendored Three.js scenes. You can follow six stages—sense, see, decide, act, report, reclaim—and inspect the tower, a sensor pod, and a controller-board model.

One important distinction: this repository contains the website and sample API routes, but its live-looking charts and terminal are demo data. The firmware, PCB design, model file, service credentials, and measurement logs are not in this repository. I would present the 95-percent water figure as the project's stated comparison until I can show the experiment data. The next step is to connect the physical evidence to the dashboard and validate every number.

Suggested live demo sequence

TIME	DO THIS	SAY THIS
0:00–0:35	Open the hero; state the problem and intent.	"I'm exploring a compact, sensor-guided way to grow while managing water and attention."
0:35–1:20	Scroll the six-stage loop and pause on DECIDE / RECLAIM.	"This shows the control logic; it is the architecture story, not a live device readout."
1:20–2:10	Use the guided tower camera chapters; orbit the scene.	"The model lets us inspect the system layout and why each tier exists."
2:10–3:00	Show sensing, board, and subsystem tabs.	"These are the components and interfaces described by the project; I'll distinguish them from what is implemented in this web repo."
3:00–4:00	Show demo telemetry/API note, then close with next steps.	"The values are sample data. The next milestone is real device integration and a reproducible water-use test."

If your physical system and live integrations are fully implemented outside the repository, adapt the explicit limitations to explain where those artifacts and logs can be inspected. Do not claim a limitation that is untrue; do show evidence for any stronger claim.

18 / EXAMINER Q&A · SYSTEM

Questions about the growing system

Short, grounded answers show that you understand both the intended design and the engineering trade-offs.

Why hydroponics rather than soil?

It makes nutrient solution controllable and recirculatable, and vertical racks can use floor area efficiently. The trade-off is dependence on pumps, power, nutrient balance, cleaning, and dependable monitoring.

What does “closed loop” mean here?

The intended fluid path sends nutrient solution from a reservoir through the grow sites and back to the reservoir. It does not mean zero loss: plants absorb water, evaporation occurs, and the reservoir still needs maintenance.

Why use an ESP8266?

It is a compact, low-cost Wi-Fi microcontroller suited to sensor acquisition and simple local control. I would also explain its memory/compute limits and prove the exact board and firmware used.

What happens if Wi-Fi goes down?

The design narrative says local irrigation/light decisions should continue; remote sync, alerts, and weather updates may pause. I would demonstrate a real offline test before calling that behavior verified.

How do you avoid overwatering?

Use calibrated root-zone thresholds, bounded pump duration, reservoir checks, and a safety timeout. A robust controller should reject implausible sensor readings and log why it acted.

How do you know the water-saving number?

It needs a fair baseline: same crop, plant count, duration, location, and conditions, with metered input/refill and a defined soil-bed comparison. The website’s 95% figure is not accompanied by logs in this repo.

What is the value of per-plant sensing?

A shared shelf timer can overwater one pot while another remains dry. Per-site sensing can make the decision reflect local need, provided the sensor is calibrated and the system can control the corresponding zone.

> **ANSWER FORMULA**
> Define the term → describe the mechanism → name one trade-off → state how you would measure or test it. That sounds more confident than giving a slogan without a validation method.

19 / EXAMINER Q&A · SOFTWARE AND EVIDENCE

Questions about the code

These answers are especially useful because the repository combines real interactive code with sample data and an aspirational hardware narrative.

Is the displayed telemetry live?

No, not from this repository. The React telemetry component uses a seeded random-walk simulation; the console uses a fixed pool of example events. The `/api/telemetry` route also generates random samples, but the UI does not call it.

Does the page use Firebase, Twilio, or OpenWeatherMap?

Those services are described by the project, but no SDK/configuration or outbound service call is in the active app. The weather route returns a fixed Delhi example; the other services appear in narrative/demo text.

Are the 3D models downloaded GLBs?

No GLB/GLTF files are present. Each active scene builds its model from Three.js geometry and materials in a standalone HTML page. The model-prompt document describes possible future/generated assets.

Why use iframes for Three.js?

They isolate the scene’s DOM and WebGL runtime from the React page. ScenePreloader mounts them once and moves their wrappers over section slots; a postMessage bridge sends camera/board commands.

Why Next.js instead of a plain HTML page?

The site needs a structured React component system, App Router metadata, static prerendering, and API route handlers in one deployment. Next.js serves the narrative and static pages efficiently.

What did you test?

`npm run verify` passed during the repository review: TypeScript, Next lint, and production build. There are no test scripts for unit, browser, API contract, or hardware-in-the-loop testing in package.json.

Where is the firmware/PCB/model?

Those artifacts are not in this repository snapshot. If they exist in a separate project or physical prototype, I would show the source, schematic, model/evaluation report, and measured results separately.

Why show demo data at all?

It lets the website demonstrate layout, trends, and interaction before a device pipeline is connected. The important improvement is to label it as simulation, provide an explicit demo/live mode, and never imply it came from sensors.

> **AVOID A TRAP ANSWER**
> Do not say “the cloud is connected” just because a card says Firebase or a route calls itself OpenWeatherMap. Point to the actual adapter, request, data source, or deployment configuration—or call it a planned integration.

20 / ROADMAP AND DEFENSE CHECKLIST

What to improve next

A practical next plan turns a compelling story into a reproducible engineering project and a safer public deployment.

PRIORITY	NEXT ACTION	WHY IT MATTERS
P0 · Trust	Update README to match the active code. Label charts, terminal, health/weather/stats routes as demo/static/synthetic until connected.	Prevents a judge or user from confusing polished presentation with live data.
P0 · Security	Upgrade Next.js from 14.2.5 to a patched compatible release; audit the full lockfile; then rerun typecheck, lint, build, and deployment tests.	The reviewed npm audit reported 1 critical + 3 high production findings, 21 total including development dependencies.
P1 · Evidence	Add water-test protocol, raw logs, sensor calibration records, a defined baseline, and uncertainty/results.	Turns “95%” from a display constant into a defensible experiment.
P1 · Embedded	Publish the firmware source, pin map, board schematic/PCB, BOM, wiring diagram, and fail-safe test plan.	Allows anyone to inspect the hardware/control claims and reproduce the device.
P1 · ML	Document model origin/license, dataset split, labels, evaluation metrics, confusion matrix, preprocessing, runtime/arena, memory use, and measured latency.	Makes disease-detection claims meaningful, safe, and technically reproducible.
P1 · Integration	Create a typed telemetry contract; connect the UI to authenticated device data; add visible live/demo mode and last-updated timestamp.	Bridges the gap between sample UI and actual system without losing trust.
P2 · Reliability	Add API schema validation, device auth, rate limits, CORS allowlist, persistence/queue, and contact delivery provider if needed.	Protects public routes before they accept real devices or personal data.
P2 · Test / performance	Add API tests, browser smoke tests, accessibility checks, and low-end device profiling; pause or defer hidden WebGL loops.	Reduces regressions and improves exhibition-device reliability.

Before a public demo

- Reconcile 95% vs 42% water claims; select one metric, baseline, and evidence.
- Reconcile 38 ms vs 340 ms, 612 KB/214 KB memory, and 91.3% accuracy; cite measured model build.
- Confirm actual sensor types (capacitive vs fork-style probe) and one sampling schedule.
- State what functions work offline and show the test; protect the pump from stuck-on and low-water conditions.
- Clearly label synthetic data, static sample endpoints, and any feature that is planned rather than connected.
- Review npm audit, keep secrets out of the repo, and verify the live deployment uses patched dependencies.
- Test the 3D tour on the actual laptop/phone and keep a non-WebGL explanation ready.

A STRONG NEXT MILESTONE

Build one end-to-end vertical slice: one real moisture sensor → one ESP8266 decision → one safe pump event → one authenticated timestamped event → one UI card. Measure it, label the data source, and document one failure test. A small verified path is stronger than a large unverified feature list.

21 / GLOSSARY AND SOURCE MAP

Terms and where the facts came from

This last page makes it easier to revisit the code, explain unfamiliar words, and separate repository facts from project claims.

TERM	PLAIN-LANGUAGE MEANING
Hydroponics	Growing plants in a nutrient-bearing water solution rather than soil.
Closed-loop recirculation	Collecting and returning a solution to a tank; it reduces drainage but does not eliminate plant uptake, evaporation, or maintenance.
Edge / on-device	Processing performed on the controller near the sensor/camera rather than sending the task to a remote server.
TensorFlow Lite / INT8	A compact inference runtime/model format; INT8 quantization stores many weights/activations with 8-bit integers to reduce size and compute.
NPK	Nitrogen, phosphorus, and potassium—three nutrients commonly tracked in growing solutions.
RS485 / Modbus	An electrical serial bus and a common device communication protocol used by industrial sensors.
PWM / MOSFET	Pulse-width modulation varies average power; a MOSFET is a transistor commonly used to switch a DC load efficiently.
Watchdog	A timer that resets a controller if software stops making expected progress.
ESP8266	A Wi-Fi-capable microcontroller family used in NodeMCU development boards.
RTDB	Firebase Realtime Database; the project narrative names it as a cloud data store.
Static prerender	HTML/data prepared during the build and served without running page logic for every request.
iframe / postMessage	An embedded browser document and a browser API for sending messages between parent and embedded page.

Repository source map

REF	FILES REVIEWED	USE IN THIS GUIDE
S1	README.md	Project mission, team, top-level claims, and older stack inventory (cross-checked for drift).
S2	app/page.tsx; app/layout.tsx	Actual page order, metadata, fonts, and App Router entry.
S3	app/components/*.tsx; app/components/site/*	Active page sections, interactions, animations, telemetry/terminal data behavior.
S4	public/scenes/*.html; public/vendor/*; model1.html–model3.html; docs/3d-model-prompts.md	3D runtime, assets, message bridge, standalone prototypes, and model-generation brief.
S5	app/api/*/route.ts	Endpoint methods, validation, payloads, static/random/sample behavior.
S6	package.json; package-lock.json; next.config.mjs; tailwind.config.ts; app/globals.css; .nvmrc; tsconfig.json	Dependencies, scripts, theme, headers, cache, Node target, and TypeScript.
S7	DEPLOYMENT.md; vercel.json; .env.example	Deployment intent, region, build settings, environment-variable notes.

Review scope: checked-out repository, 18 October 2026; `npm ci` and `npm run verify` were run. This guide does not inspect a production account, separate hardware/firmware repository, physical measurements, service credentials, or private training data. Project claims are not independently certified.

Project repository: github.com/UCHIHA-MADARA-ANUJ/Verde-CBSE · README license note: proprietary / all rights reserved.